

Manuel technique — Système de paiement Pièces

Plateforme : Pièces — Marketplace de pièces auto d'occasion **Marché :** Côte d'Ivoire (Abidjan) **Devise :** FCFA (XOF) **Version :** 2.0 — Août 2026

Changement de modèle : le séquestre est remplacé par le paiement direct.

Le circuit de référence est désormais celui-ci : l'acheteur paie au choix en ligne à la commande ou au livreur au moment de la remise ; aucun fonds n'est bloqué ; aucune pièce n'est remise sans être payée ; et dès l'encaissement, la part du vendeur lui est transmise immédiatement. C'est ce qui est annoncé aux vendeurs et aux acheteurs — voir « La bible du démarchage vendeurs ».

*Ce document reste un manuel technique, et le code implémente encore l'escrow (`EscrowTransaction` , états `HELD` → `RELEASED` / `REFUNDED`). Les sections qui décrivent le séquestre sont donc conservées et signalées par la mention « **circuit en retrait** » : elles documentent ce qui tourne aujourd'hui, pas ce que l'on promet. Toute évolution du code doit aller vers le paiement direct, jamais renforcer le séquestre.*

Table des matières

1. Vue d'ensemble du système de paiement
2. Architecture technique
3. Méthodes de paiement
4. Intégration CinetPay
5. Configuration et variables d'environnement
6. Créer une commande (DRAFT)
7. Verrouillage des prix (price snapshot)
8. Sélectionner la méthode de paiement
9. Paiement Mobile Money (Orange, MTN, Wave)
10. Paiement en espèces (COD)
11. Webhook CinetPay
12. Système escrow (séquestre) — circuit en retrait
13. Création de l'escrow
14. Libération de l'escrow
15. Remboursement de l'escrow

- 16. Auto-libération 48 heures
- 17. Machine à états commande
- 18. Intégration livraison → escrow
- 19. Calcul des montants
- 20. Gestion des devises (FCFA/XOF)
- 21. Modèles de données Prisma
- 22. Journal des événements (OrderEvent)
- 23. Endpoints API complets
- 24. Validation des données (Zod)
- 25. Gestion d'erreurs
- 26. Sécurité
- 27. Administration et dashboard
- 28. Tests unitaires
- 29. File d'attente et jobs asynchrones
- 30. État de l'implémentation
- 31. Évolutions planifiées
- 32. Référence des fichiers
- 33. FAQ technique

1. Vue d'ensemble du système de paiement

Le système de paiement de Pièces est conçu pour le marché ivoirien où le **Mobile Money** est le moyen de paiement dominant. Le modèle est le **paiement direct** : l'argent ne stationne pas chez Pièces.

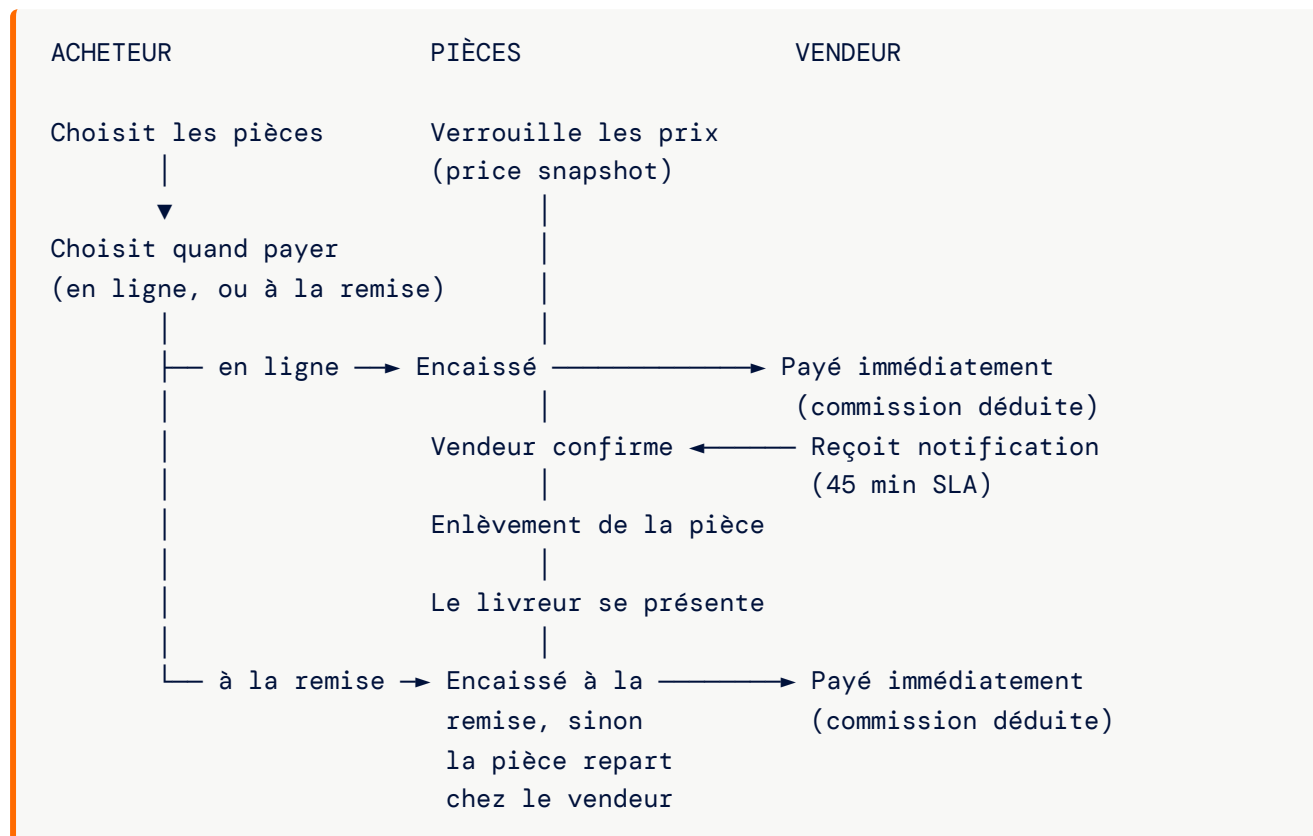
Les 3 règles du paiement direct

SYSTÈME DE PAIEMENT PIÈCES		
1. DEUX MOMENTS D'ENCAISSEMENT	2. RIEN N'EST BLOQUÉ	3. VENDEUR PAYÉ IMMÉDIATEMENT
En ligne à la commande, OU au livreur à la remise.	Aucun séquestre. Pas de pièce remise sans paiement.	Dès l'encaissement, la part du vendeur part, commission déduite.

Les deux moments d'encaissement sont **d'égale importance** : l'acheteur choisit, aucun des deux n'est un mode dégradé.

- ▶ **Paiement en ligne à la commande** — Orange Money, MTN MoMo, Wave ou carte, via CinetPay.
L'argent est encaissé avant l'enlèvement de la pièce.
- ▶ **Paiement à la remise** — espèces ou mobile money remis au livreur au moment où il donne la pièce.

Flux simplifié



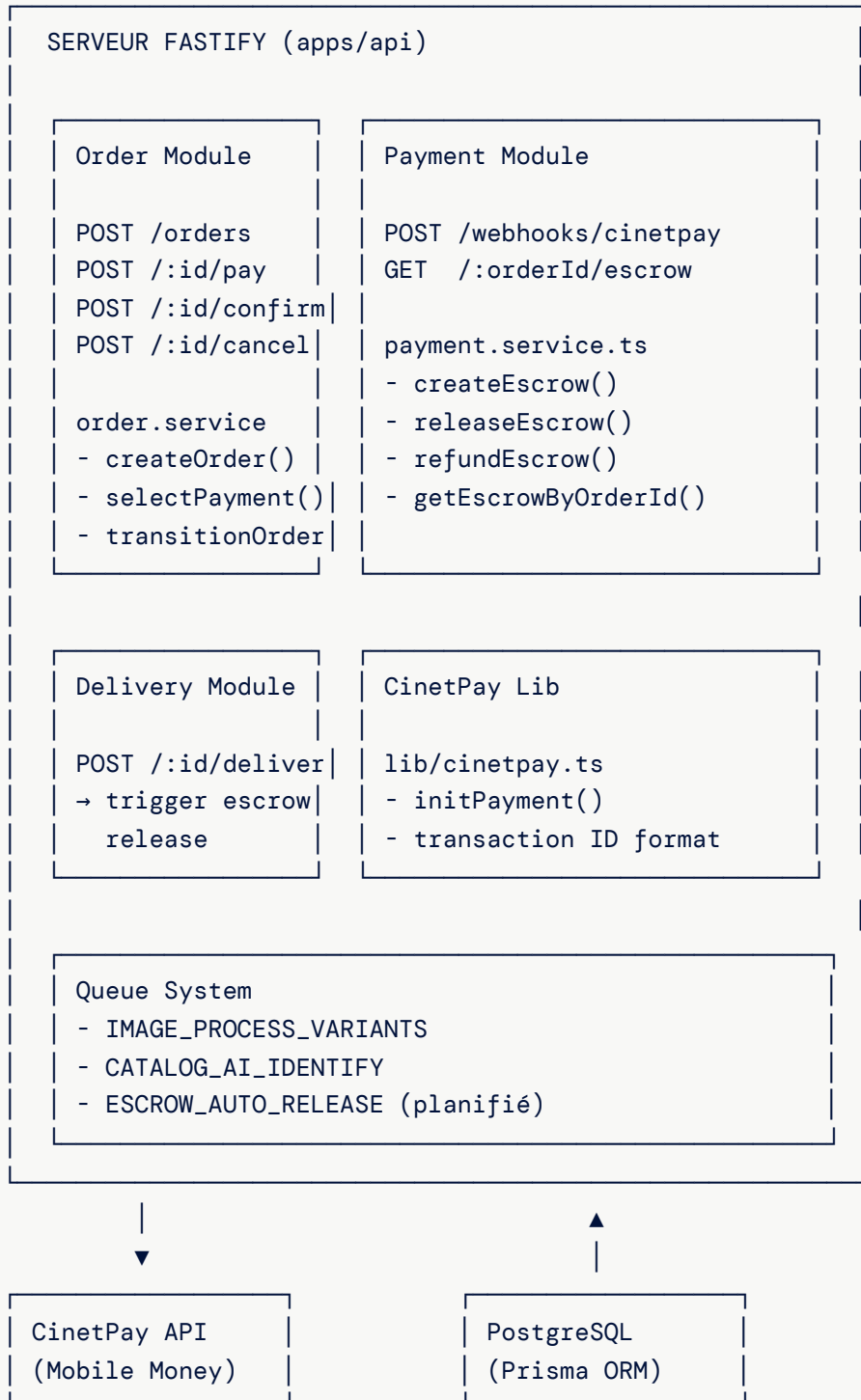
Le point non négociable : la pièce n'est jamais remise sans encaissement. S'il n'y a pas de paiement, il n'y a pas de livraison — la pièce retourne au vendeur. Il n'existe pas de troisième issue.

En cas de retour ou de litige

Sans séquestre, le remboursement n'est plus un déblocage de fonds : c'est une **reprise traitée par Pièces avec le vendeur**, dans les limites du contrat d'adhésion (retour sous 48 h si la pièce ne correspond pas à l'annonce). Quand le retour est justifié, **la commission n'est pas due**.

2. Architecture technique

2.1 Composants



2.2 Flux de données

Paieement Mobile Money :

Client → Frontend → POST /orders/:id/pay → initPayment() → CinetPay URL

Client → CinetPay page → Confirme paieement → CinetPay webhook → createEscrow()

Paieement COD :

Client → Frontend → POST /orders/:id/pay { method: COD } → PAID immédiatement

3. Méthodes de paiement

3.1 Les 4 méthodes

Méthode	Code	Type	Passerelle	Limite
Orange Money	ORANGE_MONEY	Mobile Money	CinetPay	Aucune
MTN MoMo	MTN_MOMO	Mobile Money	CinetPay	Aucune
Wave	WAVE	Mobile Money	CinetPay	Aucune
Espèces	COD	Cash on Delivery	Direct	≤ 75 000 FCFA

3.2 Comportement par méthode

Méthode	CinetPay	Escrow créé	Flux
Orange Money	Oui	Oui (HELD)	DRAFT → PENDING_PAYMENT → PAID
MTN MoMo	Oui	Oui (HELD)	DRAFT → PENDING_PAYMENT → PAID
Wave	Oui	Oui (HELD)	DRAFT → PENDING_PAYMENT → PAID
COD	Non	Non	DRAFT → PAID (immédiat)

3.3 Enum Prisma

```
enum PaymentMethod {  
  ORANGE_MONEY  
  MTN_MOMO  
  WAVE  
  COD  
}
```

4. Intégration CinetPay

4.1 Présentation

CinetPay est la passerelle de paiement Mobile Money utilisée par Pièces. Elle gère les transactions Orange Money, MTN MoMo et Wave en Côte d'Ivoire.

4.2 Fonction d'initialisation

```
// apps/api/src/lib/cinetpay.ts

export async function initPayment(params: {
  amount: number;           // Montant en FCFA (entier)
  orderId: string;          // ID de la commande
  description: string;      // Description du paiement
  customerPhone: string;    // Téléphone du client (+225...)
  paymentMethod: string;    // ORANGE_MONEY | MTN_MOMO | WAVE
}): Promise<PaymentInitResult>
```

4.3 Requête vers CinetPay

```
{
  "apikey": "${CINETPAY_API_KEY}",
  "site_id": "${CINETPAY_SITE_ID}",
  "transaction_id": "pieces_${orderId}_${Date.now()}",
  "amount": 45000,
  "currency": "XOF",
  "description": "Auto parts order",
  "customer_phone_number": "+22507000000000",
  "channels": "ORANGE_CI",
  "return_url": "${NEXT_PUBLIC_URL}/orders/success",
  "notify_url": "${API_URL}/api/v1/webhooks/cinetpay"
}
```

4.4 Format de l'identifiant de transaction

pieces_{orderId}_{timestamp}

Exemple : pieces_order-abc123_1709300400000

Décomposition :

pieces	→ préfixe fixe
order-abc123	→ ID de la commande
1709300400000	→ timestamp Unix en ms

4.5 Résultat de l'initialisation

```
interface PaymentInitResult {
  transactionId: string; // "pieces_order-abc123_1709300400000"
  paymentUrl: string | null; // URL CinetPay pour le client
  status: 'pending' | 'error';
}
```

4.6 Mapping des canaux CinetPay

Méthode Pièces	Canal CinetPay
ORANGE_MONEY	ORANGE_CI
MTN_MOMO	MTN_CI
WAVE	ALL

4.7 Mode développement (stub)

Quand les variables CinetPay ne sont pas configurées :

```
// Retourne un mock
{
  transactionId: `txn_${orderId}_${Date.now()}`,
  paymentUrl: null,
  status: 'pending'
}
```

5. Configuration et variables d'environnement

5.1 Variables CinetPay

```
CINETPAY_API_KEY=<clé API CinetPay>
CINETPAY_SITE_ID=<ID du site CinetPay>
```

5.2 Variables URL

```
API_URL=https://api.pieces.ci # URL de l'API (pour notify_url)
NEXT_PUBLIC_URL=https://pieces.ci # URL du frontend (pour return_url)
```

5.3 Comportement sans configuration

Variable manquante	Comportement
CINETPAY_API_KEY	Mode stub, paiements simulés
CINETPAY_SITE_ID	Mode stub, paiements simulés
API_URL	Webhook URL non fonctionnel
NEXT_PUBLIC_URL	Redirection post-paiement non fonctionnelle

6. Créer une commande (DRAFT)

6.1 Endpoint

```
POST /api/v1/orders
Auth: Bearer Token (rôle MECHANIC ou SELLER)
```

6.2 Corps de la requête

```
{
  "items": [
    { "catalogItemId": "catalog-item-uuid-1" },
    { "catalogItemId": "catalog-item-uuid-2" }
  ],
  "ownerPhone": "+22507000000000",
  "laborCost": 15000
}
```

Champ	Type	Obligatoire	Description
items	Array	Oui	Min 1 article avec catalogItemId
ownerPhone	String	Non	Téléphone du propriétaire (+225...)
laborCost	Number	Non	Frais de main d'œuvre mécanicien (FCFA)

6.3 Processus de création

1. Valider les articles (existent, publiés, en stock, vendeur actif)
2. Verrouiller les prix (priceSnapshot = prix actuel)
3. Calculer le montant total
4. Générer un shareToken unique
5. Créer la commande en statut DRAFT
6. Enregistrer un OrderEvent (→ DRAFT)

6.4 Réponse

```
{
  "data": {
    "id": "order-uuid",
    "status": "DRAFT",
    "totalAmount": 53500,
    "laborCost": 15000,
    "deliveryFee": 0,
    "shareToken": "abc123xyz789",
    "items": [
      {
        "name": "Alternateur Toyota Corolla",
        "priceSnapshot": 45000,
        "vendorShopName": "Auto Parts Abidjan"
      },
      {
        "name": "Filtre à huile",
        "priceSnapshot": 8500,
        "vendorShopName": "Pièces Express"
      }
    ]
  }
}
```

7. Verrouillage des prix (price snapshot)

7.1 Principe

Quand une commande est créée, le prix de chaque article est **verrouillé** (snapshot). Même si le vendeur modifie son prix ensuite, la commande conserve le prix initial.

7.2 Implémentation

```
// Dans createOrder()
const orderItem = {
  catalogItemId: catalogItem.id,
  priceSnapshot: catalogItem.price,    // ← Prix figé ici
  vendorId: catalogItem.vendorId,
  vendorShopName: catalogItem.vendor.shopName,
  name: catalogItem.name,
  category: catalogItem.category,
  imageThumbUrl: catalogItem.images?.[0]?.url
};
```

7.3 Modèle de données

```
model OrderItem {
  id          String    @id @default(uuid())
  orderId     String
  catalogItemId String
  vendorId    String
  vendorShopName String
  name        String
  category    String?
  priceSnapshot Int      // ← PRIX VERROUILLÉ
  quantity    Int        @default(1)
  imageThumbUrl String?
}
```

7.4 Pourquoi le verrouillage ?

Sans verrouillage	Avec verrouillage
Le vendeur modifie le prix → le client paie plus	Le prix est fixé au moment de la commande
Litige possible sur le montant	Montant garanti pour le client
Incohérence entre commande et paiement	Cohérence totale

8. Sélectionner la méthode de paiement

8.1 Endpoint

```
POST /api/v1/orders/{orderId}/pay
Auth: Aucune (accessible via lien partagé)
```

8.2 Corps de la requête

```
{
  "paymentMethod": "ORANGE_MONEY"
}
```

Valeurs acceptées: `ORANGE_MONEY`, `MTN_MOMO`, `WAVE`, `COD`

8.3 Logique de traitement

```
export async function selectPaymentMethod(
  orderId: string,
  paymentMethod: string,
  actor: string
) {
  const order = await prisma.order.findUnique({ where: { id: orderId } });

  // 1. Vérifier que la commande est en DRAFT
  if (order.status !== 'DRAFT') {
    throw new AppError('ORDER_INVALID_STATUS', 400);
  }

  // 2. Vérifier la limite COD
  if (paymentMethod === 'COD' && order.totalAmount > 75_000) {
    throw new AppError('ORDER_COD_LIMIT', 400);
  }

  // 3. Déterminer le nouveau statut
  const toStatus = paymentMethod === 'COD' ? 'PAID' : 'PENDING_PAYMENT';

  // 4. Mettre à jour la commande
  // 5. Si Mobile Money : appeler initPayment() → retourner paymentUrl
  // 6. Si COD : marquer comme PAID avec paidAt = now
}
```

8.4 Réponse (Mobile Money)

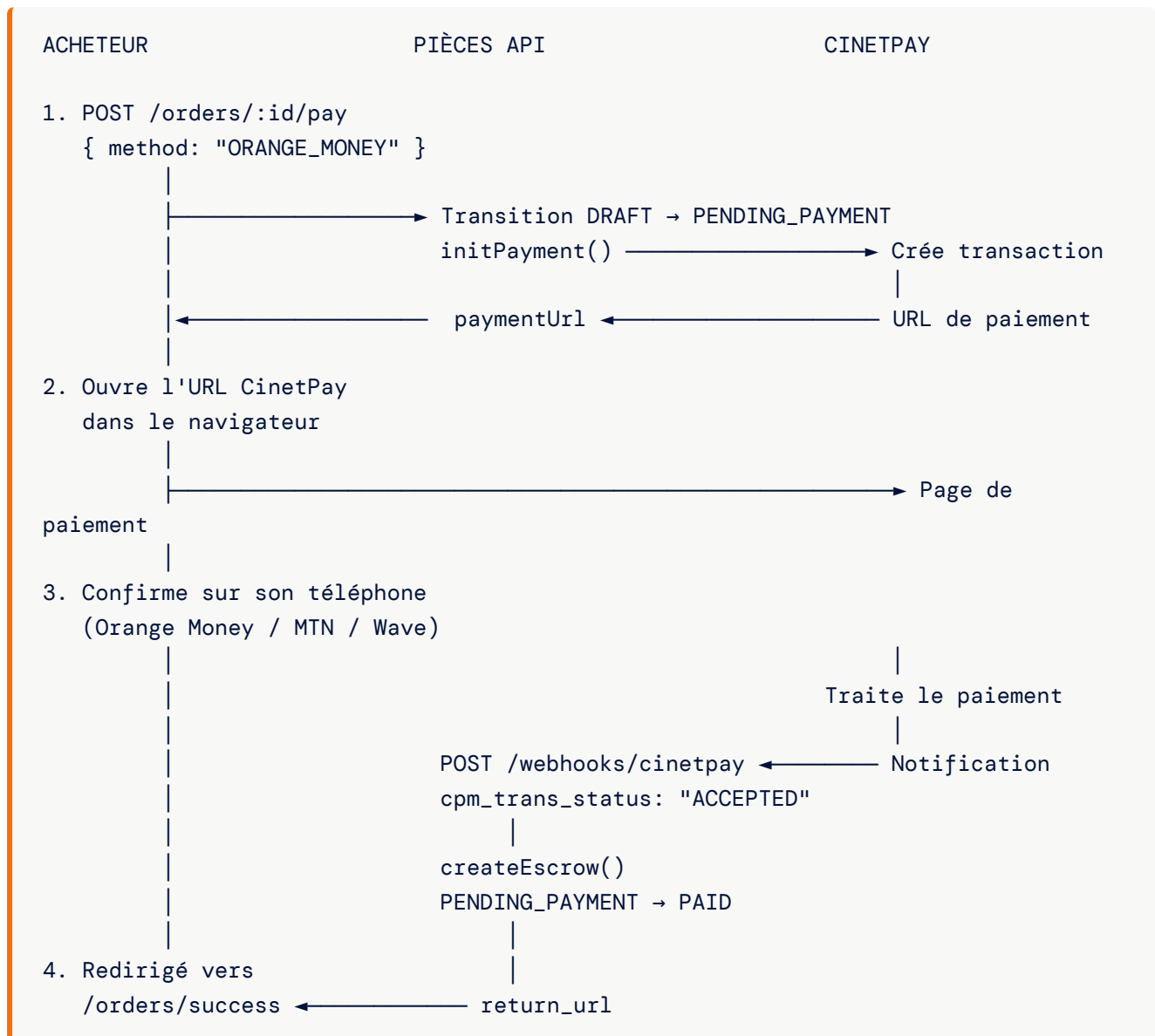
```
{
  "data": {
    "orderId": "order-uuid",
    "status": "PENDING_PAYMENT",
    "paymentUrl": "https://checkout.cinetpay.com/payment/abc123"
  }
}
```

8.5 Réponse (COD)

```
{
  "data": {
    "orderId": "order-uuid",
    "status": "PAID",
    "paidAt": "2026-03-01T10:30:00Z"
  }
}
```

9. Paiement Mobile Money (Orange, MTN, Wave)

9.1 Flux complet



9.2 Identifiant de transaction

Le `transaction_id` envoyé à CinetPay est au format :

```
pieces_{orderId}_{timestamp}
```

Ce format permet de retrouver la commande correspondante dans le webhook.

10. Paiement en espèces (COD)

10.1 Principe

Le paiement en espèces (Cash on Delivery) permet au client de payer directement au livreur lors de la réception des pièces.

10.2 Limite

MONTANT MAXIMUM COD : 75 000 FCFA

Au-delà de ce montant, le client doit utiliser un paiement Mobile Money.

10.3 Flux COD



10.4 Pas d'escrow pour COD — c'est le modèle cible

Le paiement à la remise **ne crée pas de transaction escrow** : l'encaissement se fait directement entre l'acheteur et le livreur, et la part du vendeur lui est transmise immédiatement, commission déduite.

Ce comportement n'est plus une exception : **c'est le modèle vers lequel tout le système converge**. Le paiement en ligne doit s'aligner dessus — encaisser, puis transmettre au vendeur sans blocage — et non l'inverse.

11. Webhook CinetPay

11.1 Endpoint

```
POST /api/v1/webhooks/cinetpay
Auth: Aucune (appelé par les serveurs CinetPay)
```

11.2 Payload reçu

```
{
  "cpm_trans_id": "pieces_order-abc123_1709300400000",
  "cpm_trans_status": "ACCEPTED",
  "cpm_amount": "45000",
  "cpm_site_id": "123456"
}
```

11.3 Champs importants

Champ	Description
cpm_trans_id	Identifiant de transaction (contient l'orderId)
cpm_trans_status	Statut: ACCEPTED , REFUSED , CANCELLED
cpm_amount	Montant payé (chaîne de caractères)
cpm_site_id	Identifiant du site CinetPay

11.4 Traitement

```
// 1. Extraire l'orderId du transaction_id
const parts = cpm_trans_id.split('_');
const orderId = parts[1]; // "order-abc123"

// 2. Si le paiement est accepté
if (cpm_trans_status === 'ACCEPTED') {
  // 3. Créer l'escrow
  await createEscrow(orderId, parseInt(cpm_amount));

  // 4. Transition : PENDING_PAYMENT → PAID
  await transitionOrder(orderId, 'PAID', 'system', 'Paiement CinetPay confirmé');
}

// 5. Toujours retourner 200 OK
return { status: 'ok' };
```

11.5 Réponses HTTP

Code	Cas
200	Webhook traité (même si paiement refusé)
400	<code>cpm_trans_id</code> manquant ou invalide

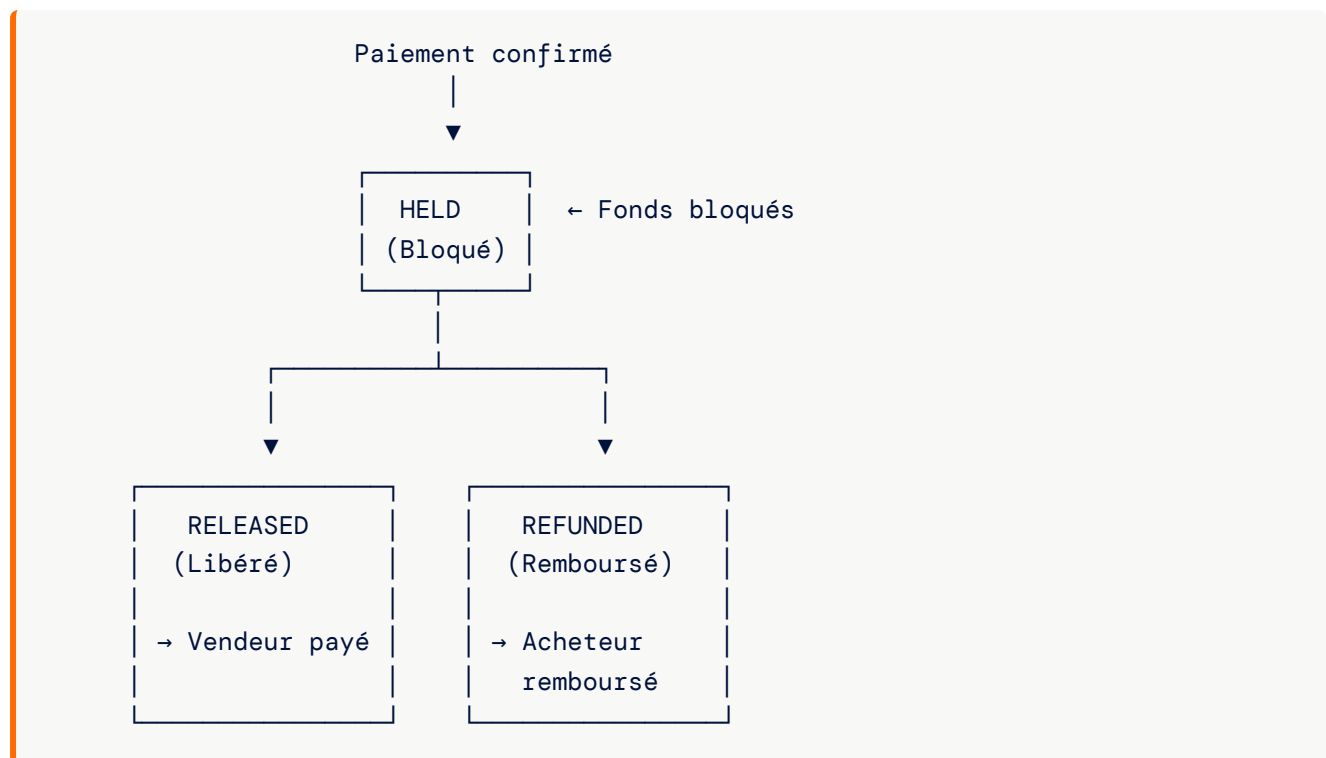
12. Système escrow (séquestre) — circuit en retrait

Circuit en retrait. Les sections 12 à 16 et 18.2 décrivent le séquestre tel qu'il est **encore implémenté dans le code**. Ce n'est plus le modèle annoncé : le blocage des fonds est remplacé par le paiement direct décrit en section 1. Elles restent ici parce qu'un développeur doit pouvoir lire ce qui tourne aujourd'hui — pas parce qu'il faut le maintenir.

12.1 Principe

L'**escrow** (séquestre) bloquait les fonds entre le paiement et la livraison : le vendeur ne recevait le paiement qu'après confirmation de la livraison. Dans le modèle actuel, cette attente disparaît — le vendeur est payé dès l'encaissement, et la protection de l'acheteur repose sur le fait qu'**aucune pièce n'est remise sans être payée**, pas sur la rétention de son argent.

12.2 Les 3 états



12.3 Quand chaque état est utilisé

État	Déclencheur	Résultat
HELD	Webhook CinetPay (paiement accepté)	Fonds en attente
RELEASED	Livraison confirmée + 48h	Vendeur reçoit le paiement
REFUNDED	Annulation ou litige en faveur de l'acheteur	Acheteur remboursé

13. Création de l'escrow — circuit en retrait

13.1 Fonction

```
export async function createEscrow(orderId: string, amount: number) {
  return prisma.escrowTransaction.create({
    data: {
      orderId,           // Lien vers la commande (unique)
      amount,            // Montant en FCFA
      status: 'HELD',    // État initial
      heldAt: new Date() // Horodatage
    }
  });
}
```

13.2 Déclencheur

Appelé par le webhook CinetPay quand `cpm_trans_status === 'ACCEPTED'` .

13.3 Contraintes

- ▶ **Un seul escrow par commande** (relation 1:1, `orderId` est unique)
- ▶ Le montant correspond au `cpm_amount` reçu de CinetPay
- ▶ L'horodatage `heldAt` est automatique (`DateTime @default(now())`)

14. Libération de l'escrow — circuit en retrait

14.1 Fonction

```
export async function releaseEscrow(orderId: string) {
  const escrow = await prisma.escrowTransaction.findUnique({
    where: { orderId }
  });

  if (!escrow) throw new AppError('ESCROW_NOT_FOUND', 404);
  if (escrow.status !== 'HELD') throw new AppError('ESCROW_ALREADY_PROCESSED',
    400);

  return prisma.escrowTransaction.update({
    where: { orderId },
    data: {
      status: 'RELEASED',
      releasedAt: new Date()
    }
  });
}
```

14.2 Déclencheurs

Scénario	Déclencheur
Confirmation manuelle	L'acheteur confirme la réception dans l'app
Auto-libération	48h après la livraison (planifié, pas encore implémenté)
Résolution litige	Admin résout le litige en faveur du vendeur

14.3 Ce que signifie la libération

- ▶ Le vendeur **reçoit le paiement**
- ▶ La commande passe en statut **COMPLETED**
- ▶ L'acheteur peut toujours évaluer le vendeur

15. Remboursement de l'escrow — circuit en retrait

15.1 Fonction

```
export async function refundEscrow(orderId: string) {
  const escrow = await prisma.escrowTransaction.findUnique({
    where: { orderId }
  });

  if (!escrow) throw new AppError('ESCROW_NOT_FOUND', 404);
  if (escrow.status !== 'HELD') throw new AppError('ESCROW_ALREADY_PROCESSED', 400);

  return prisma.escrowTransaction.update({
    where: { orderId },
    data: {
      status: 'REFUNDED',
      refundedAt: new Date()
    }
  });
}
```

15.2 Déclencheurs

Scénario	Déclencheur
Annulation commande	Commande annulée avant confirmation vendeur
Litige acheteur	Admin résout le litige en faveur de l'acheteur (RESOLVED_BUYER)

15.3 Ce que signifie le remboursement

- ▶ L'acheteur **récupère ses fonds**
- ▶ La commande passe en statut **CANCELLED** (si annulation)
- ▶ Le vendeur ne reçoit **rien**

16. Auto-libération 48 heures — circuit en retrait

16.1 Principe

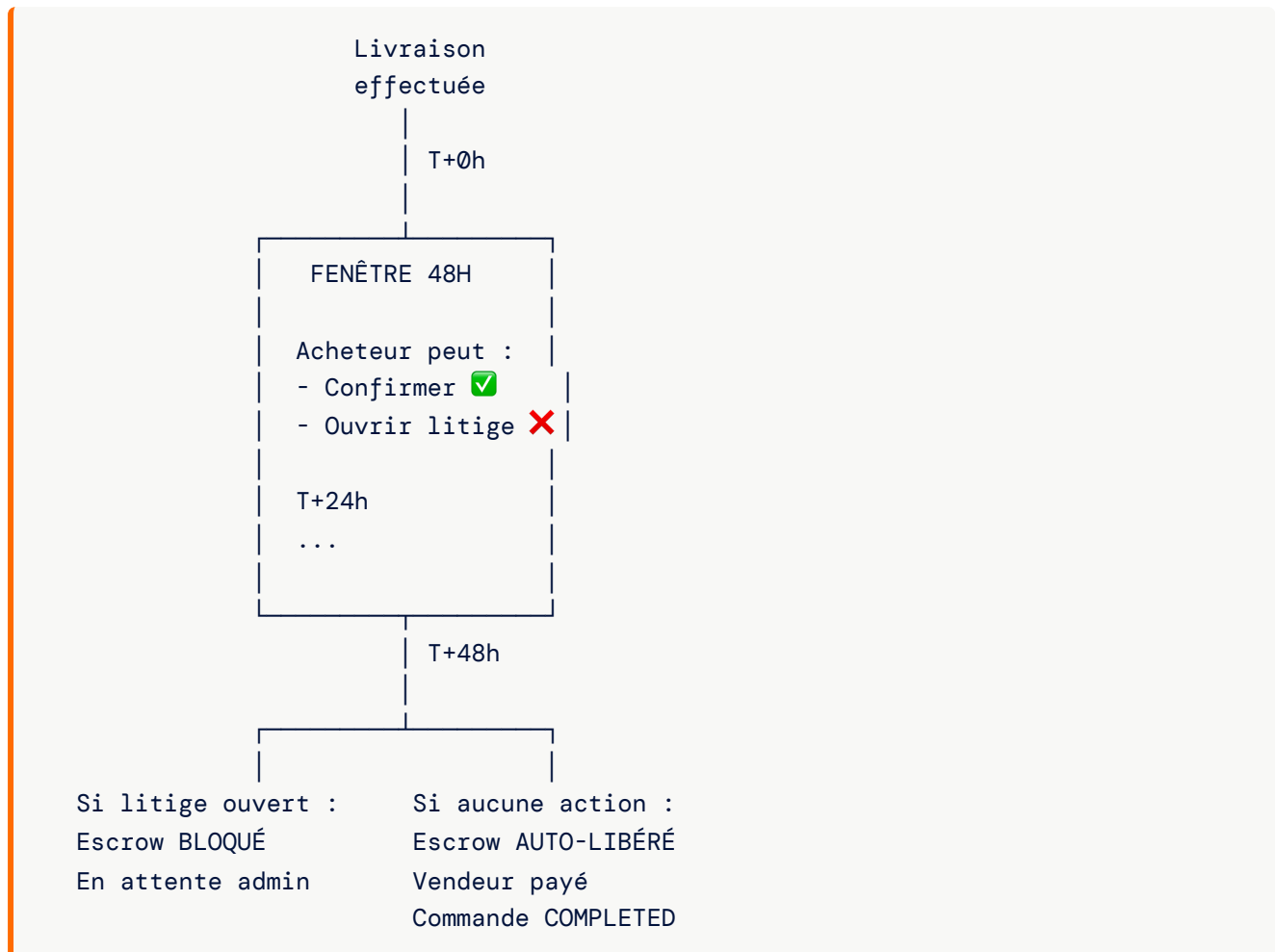
Après la livraison, l'acheteur dispose de **48 heures** pour :

- ▶ Vérifier la pièce reçue
- ▶ Confirmer la réception

- ▶ Ouvrir un litige si nécessaire

Si aucune action n'est prise, les fonds sont **automatiquement libérés** au vendeur.

16.2 Diagramme temporel



16.3 État de l'implémentation

La machine à états permet la transition **DELIVERED → COMPLETED**, mais le **job schedulé pour l'auto-libération après 48h n'est pas encore implémenté**.

16.4 Implémentation planifiée

```
// 1. Quand la livraison est confirmée, enqueue un job
await enqueue('ESCROW_AUTO_RELEASE', {
  orderId: delivery.orderId
}, {
  scheduledAt: new Date(Date.now() + 48 * 60 * 60 * 1000) // +48h
});

// 2. Handler du job
async function handleEscrowAutoRelease(job) {
  const { orderId } = job.payload;

  // Vérifier qu'aucun litige n'a été ouvert
  const dispute = await prisma.dispute.findFirst({
    where: { orderId, status: 'OPEN' }
  });

  if (dispute) {
    // Litige ouvert → ne pas libérer
    return;
  }

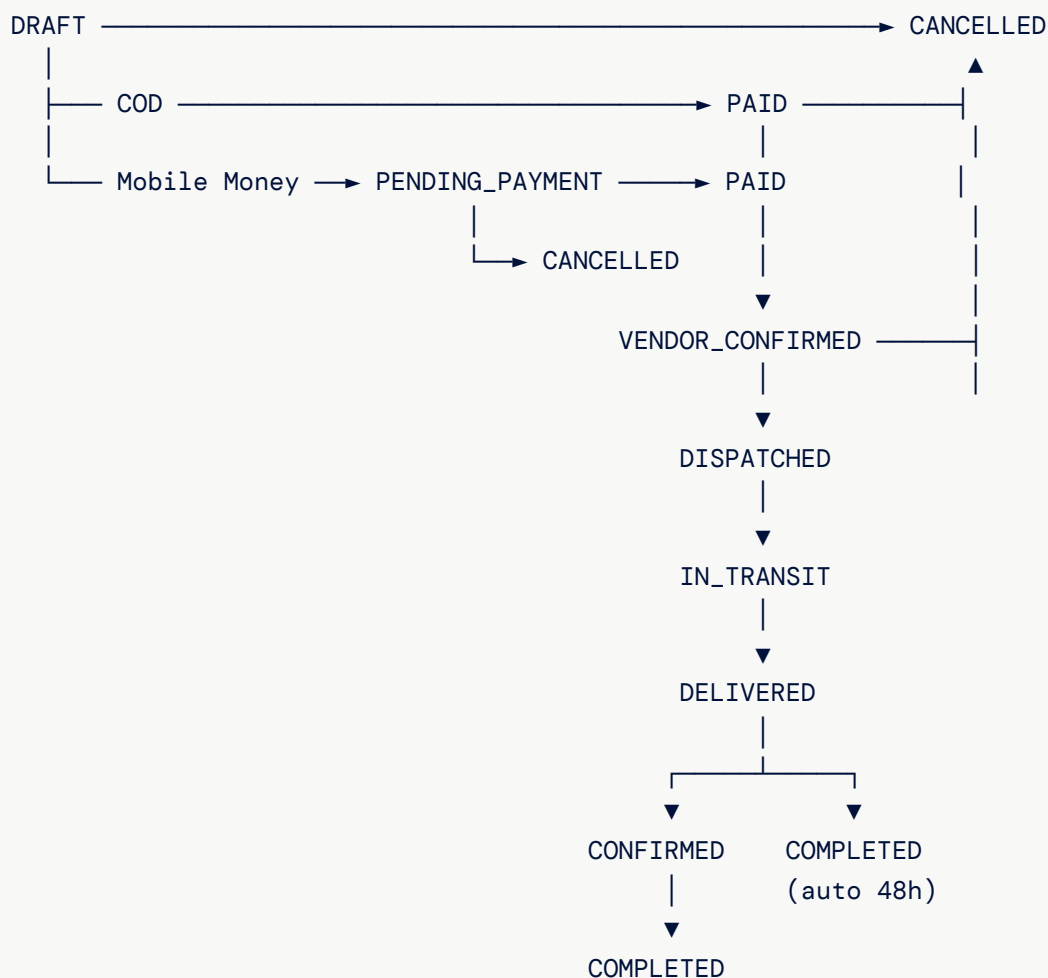
  // Libérer l'escrow
  await releaseEscrow(orderId);
  await transitionOrder(orderId, 'COMPLETED', 'system', 'Auto-released after
    48h');
}
```

17. Machine à états commande

17.1 Transitions valides

```
const VALID_TRANSITIONS = {  
  DRAFT:      ['PENDING_PAYMENT', 'PAID', 'CANCELLED'],  
  PENDING_PAYMENT: ['PAID', 'CANCELLED'],  
  PAID:        ['VENDOR_CONFIRMED', 'CANCELLED'],  
  VENDOR_CONFIRMED: ['DISPATCHED', 'CANCELLED'],  
  DISPATCHED:  ['IN_TRANSIT'],  
  IN_TRANSIT:  ['DELIVERED'],  
  DELIVERED:   ['CONFIRMED', 'COMPLETED'],  
  CONFIRMED:   ['COMPLETED'],  
  COMPLETED:  [],      // Terminal  
  CANCELLED:   [],      // Terminal  
};
```

17.2 Diagramme complet



17.3 Transitions liées au paiement

Transition	Déclencheur	Impact escrow
DRAFT → PENDING_PAYMENT	Sélection Mobile Money	Aucun
DRAFT → PAID	Sélection COD	Aucun (pas d'escrow)
PENDING_PAYMENT → PAID	Webhook CinetPay	Escrow créé (HELD)
DELIVERED → CONFIRMED	Confirmation manuelle	Escrow libéré (RELEASED)
DELIVERED → COMPLETED	Auto-libération 48h	Escrow libéré (RELEASED)
* → CANCELLED	Annulation	Escrow remboursé (REFUNDED)

17.4 Fonctions helper

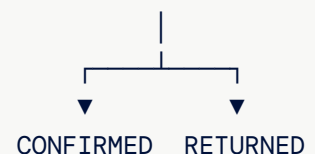
```
// Vérifie si la transition est valide
export function canTransition(from: OrderStatus, to: OrderStatus): boolean

// Retourne les transitions possibles depuis un état
export function getValidTransitions(from: OrderStatus): OrderStatus[]
```

18. Intégration livraison → escrow

18.1 Machine à états livraison

PENDING_ASSIGNMENT → ASSIGNED → PICKUP_IN_PROGRESS → IN_TRANSIT → DELIVERED



18.2 Déclenchement de la libération escrow

```
Livreur confirme la livraison :  
  POST /api/v1/deliveries/{deliveryId}/deliver  
  
  |  
  ▼  
  Delivery { status: 'DELIVERED', deliveredAt: now }  
  Order { status: 'DELIVERED' }  
  |  
  └─ Si confirmation manuelle par l'acheteur :  
      Order → CONFIRMED → COMPLETED  
      Escrow → RELEASED  
  |  
  └─ Si aucune action après 48h (job planifié) :  
      Order → COMPLETED  
      Escrow → RELEASED
```

18.3 Cas spécial : client absent

```
Livreur marque "Client absent" :  
  POST /api/v1/deliveries/{deliveryId}/client-absent  
  
  Delivery { clientAbsent: true }  
  → Nouvelle tentative de livraison organisée  
  → Escrow reste HELD
```

18.4 Cas spécial : retour

```
Livraison retournée :  
  Delivery { status: 'RETURNED' }  
  → Escrow devrait passer à REFUNDED  
  → Commande → CANCELLED
```

19. Calcul des montants

19.1 Composantes du prix

```
TOTAL COMMANDE =  $\Sigma(\text{priceSnapshot} \times \text{quantity}) + \text{laborCost} + \text{deliveryFee}$ 
```

Où :

```
priceSnapshot = Prix de chaque pièce au moment de la commande  
quantity      = Quantité (par défaut : 1)  
laborCost     = Frais de main d'œuvre mécanicien (optionnel)  
deliveryFee   = Frais de livraison (non encore calculé, = 0)
```

19.2 Exemple de calcul

Article 1 : Alternateur Toyota	45 000 FCFA × 1 =	45 000 FCFA
Article 2 : Filtre à huile	8 500 FCFA × 1 =	8 500 FCFA
Sous-total pièces :		53 500 FCFA
Main d'œuvre mécanicien :		15 000 FCFA
Frais de livraison :		0 FCFA
TOTAL :		68 500 FCFA

19.3 Implémentation

```
// Dans createOrder()  
const totalAmount = catalogItems.reduce(  
  (sum, item) => sum + (item.price ?? 0), 0  
);  
  
// Le laborCost est ajouté séparément dans le modèle Order  
// Le deliveryFee est toujours 0 en v1
```

19.4 Note sur les frais de livraison

Le champ `deliveryFee` existe dans le modèle mais vaut **toujours 0** en v1. Le calcul basé sur le mode de livraison (EXPRESS/STANDARD) et la zone est planifié pour une version future.

20. Gestion des devises (FCFA/XOF)

20.1 Devise unique

Pièces opère exclusivement en **Franç CFA** (FCFA), code ISO : **XOF** (West African CFA Franc).

20.2 Stockage

Aspect	Détail
Type Prisma	Int (entier)
Pas de décimales	Le FCFA n'a pas de centimes
Stockage en base	45000 (pas 45000.00)
API CinetPay	currency: "XOF"

20.3 Formatage côté client

```
function formatCfa(amount: number): string {
  return new Intl.NumberFormat('fr-CI', {
    style: 'currency',
    currency: 'XOF',
    minimumFractionDigits: 0,
    maximumFractionDigits: 0
  }).format(amount);
}
```

```
// Exemples :
formatCfa(45000)    // → "45 000 F CFA"
formatCfa(75000)    // → "75 000 F CFA"
formatCfa(1500000)  // → "1 500 000 F CFA"
```

20.4 Taux de conversion (référence)

Devise	Taux indicatif
1 EUR	~655,957 FCFA (taux fixe)
1 USD	~600 FCFA (variable)

21. Modèles de données Prisma

21.1 Order

```
model Order {
  id          String      @id @default(uuid())
  initiatorId String      @map("initiator_id")
  initiator    User        @relation("OrderInitiator", fields:
[initiatorId])
  ownerPhone   String?     @map("owner_phone")
  status       OrderStatus @default(DRAFT)
  paymentMethod PaymentMethod? @map("payment_method")
  shareToken   String      @unique @map("share_token")
  totalAmount  Int         @default(0) @map("total_amount")
  deliveryFee  Int         @default(0) @map("delivery_fee")
  laborCost    Int?        @map("labor_cost")
  vendorConfirmedAt DateTime? @map("vendor_confirmed_at")
  paidAt       DateTime?   @map("paid_at")
  cancelledAt  DateTime?   @map("cancelled_at")
  createdAt    DateTime    @default(now()) @map("created_at")
  updatedAt    DateTime    @updatedAt @map("updated_at")

  items      OrderItem[]
  events      OrderEvent[]
  escrow      EscrowTransaction?
  delivery    Delivery?
  sellerReviews SellerReview[]
  disputes    Dispute[]

  @@map("orders")
  @@index([initiatorId])
  @@index([status])
}
```

21.2 OrderItem

```
model OrderItem {
  id          String    @id @default(uuid())
  orderId     String    @map("order_id")
  order       Order     @relation(fields: [orderId])
  catalogItemId String  @map("catalog_item_id")
  vendorId    String    @map("vendor_id")
  vendorShopName String  @map("vendor_shop_name")
  name        String
  category    String?
  priceSnapshot Int      @map("price_snapshot")
  quantity    Int        @default(1)
  imageThumbUrl String?  @map("image_thumb_url")
  createdAt   DateTime   @default(now()) @map("created_at")

  @@map("order_items")
  @@index([orderId])
}
```

21.3 EscrowTransaction

```
model EscrowTransaction {
  id          String    @id @default(uuid())
  orderId     String    @unique @map("order_id")
  order       Order     @relation(fields: [orderId])
  amount      Int
  status      EscrowStatus @default(HELD)
  heldAt      DateTime   @default(now()) @map("held_at")
  releasedAt  DateTime?  @map("released_at")
  refundedAt  DateTime?  @map("refunded_at")

  @@map("escrow_transactions")
}
```

21.4 OrderEvent

```
model OrderEvent {
  id          String      @id @default(uuid())
  orderId     String      @map("order_id")
  order       Order       @relation(fields: [orderId])
  fromStatus  OrderStatus? @map("from_status")
  toStatus    OrderStatus  @map("to_status")
  actor       String?
  note        String?
  createdAt   DateTime     @default(now()) @map("created_at")

  @@map("order_events")
  @@index([orderId])
}
```

21.5 Enums

```
enum OrderStatus {
  DRAFT
  PENDING_PAYMENT
  PAID
  VENDOR_CONFIRMED
  DISPATCHED
  IN_TRANSIT
  DELIVERED
  CONFIRMED
  COMPLETED
  CANCELLED
}

enum PaymentMethod {
  ORANGE_MONEY
  MTN_MOMO
  WAVE
  COD
}

enum EscrowStatus {
  HELD
  RELEASED
  REFUNDED
}
```

22. Journal des événements (OrderEvent)

22.1 Principe

Chaque transition d'état de la commande est enregistrée dans la table `OrderEvent`. C'est un **journal d'audit** complet et immuable.

22.2 Exemples d'événements

#	fromStatus	toStatus	actor	note
1	null	DRAFT	user-123	Commande créée
2	DRAFT	PENDING_PAYMENT	user-456	Paiement sélectionné : ORANGE_MONEY
3	PENDING_PAYMENT	PAID	system	Paiement CinetPay confirmé
4	PAID	VENDOR_CONFIRMED	vendor-789	Vendeur a confirmé
5	VENDOR_CONFIRMED	DISPATCHED	admin-001	Livraison créée
6	DISPATCHED	IN_TRANSIT	rider-002	Livreur en route
7	IN_TRANSIT	DELIVERED	rider-002	Livré au client
8	DELIVERED	COMPLETED	system	Auto-released after 48h

22.3 Utilisation

Le journal permet :

- ▶ **Traçabilité** : Qui a fait quoi et quand
- ▶ **Conformité** : Audit trail pour les régulateurs
- ▶ **Débugage** : Comprendre les problèmes de transition
- ▶ **Analytics** : Mesurer les délais entre chaque étape

23. Endpoints API complets

23.1 Paiement

Méthode	Endpoint	Auth	Description
POST	/api/v1/webhooks/cinetpay	Non	Webhook CinetPay
GET	/api/v1/orders/{orderId}/escrow	Oui	État de l'escrow

23.2 Commandes

Méthode	Endpoint	Auth	Description
POST	/api/v1/orders	Oui	Créer une commande
GET	/api/v1/orders	Oui	Lister ses commandes
GET	/api/v1/orders/{orderId}	Oui	Détail d'une commande
GET	/api/v1/orders/share/{shareToken}	Non	Commande partagée
POST	/api/v1/orders/{orderId}/pay	Non	Sélectionner le paiement
POST	/api/v1/orders/{orderId}/confirm	Oui (SELLER)	Confirmer (vendeur)
POST	/api/v1/orders/{orderId}/cancel	Non	Annuler
GET	/api/v1/orders/history	Oui	Historique paginé

23.3 Livraison (impact escrow)

Méthode	Endpoint	Auth	Description
POST	/api/v1/deliveries	Oui (ADMIN)	Créer une livraison
POST	/api/v1/deliveries/{id}/assign	Oui (ADMIN)	Assigner un livreur
POST	/api/v1/deliveries/{id}/pickup	Oui (RIDER)	Début ramassage
POST	/api/v1/deliveries/{id}/transit	Oui (RIDER)	En transit
POST	/api/v1/deliveries/{id}/deliver	Oui (RIDER)	Livré → impact escrow
GET	/api/v1/deliveries/order/{orderId}	Non	Suivi public

23.4 Administration

Méthode	Endpoint	Auth	Description
GET	/api/v1/admin/dashboard	Oui (ADMIN)	Statistiques globales
GET	/api/v1/admin/orders	Oui (ADMIN)	Toutes les commandes

24. Validation des données (Zod)

24.1 Création de commande

```
export const createOrderSchema = z.object({
  items: z.array(z.object({
    catalogItemId: z.string().min(1),
  })).min(1, 'Au moins un article est requis'),
  ownerPhone: z.string()
    .regex(/^+225\d{10}$/, 'Numéro ivoirien requis (+225...)')
    .optional(),
  laborCost: z.number().int().min(0).optional(),
});
```

24.2 Sélection du paiement

```
export const confirmOrderSchema = z.object({
  paymentMethod: z.enum(['ORANGE_MONEY', 'MTN_MOMO', 'WAVE', 'COD']),
});
```

24.3 Annulation

```
export const cancelOrderSchema = z.object({
  reason: z.string().max(500).optional(),
});
```

25. Gestion d'erreurs

25.1 Codes d'erreur paiement

Code	HTTP	Message
ORDER_COD_LIMIT	400	Le paiement à la livraison est limité à 75,000 FCFA
ORDER_INVALID_STATUS	400	La commande n'est plus en brouillon
ESCROW_NOT_FOUND	404	Séquestre introuvable
ESCROW_ALREADY_PROCESSED	400	Séquestre déjà traité
ORDER_NOT_FOUND	404	Commande introuvable
INVALID_TRANSITION	409	Transition d'état invalide

25.2 Codes HTTP utilisés

Code	Usage
200	Webhook traité, requête réussie
201	Commande créée
400	Validation échouée, limite COD, escrow déjà traité
401	Token manquant ou invalide
403	Rôle insuffisant
404	Commande ou escrow introuvable
409	Transition d'état invalide

26. Sécurité

26.1 Authentification des endpoints

Endpoint	Auth	Justification
POST /webhooks/cinetpay	Non	Appelé par CinetPay
GET /orders/share/:token	Non	Lien partagé propriétaire
POST /orders/:id/pay	Non	Paielement via lien partagé
POST /orders/:id/cancel	Non	Annulation via lien partagé
GET /orders/:id/escrow	Oui	Données sensibles
POST /orders	Oui	Traçabilité
POST /orders/:id/confirm	Oui (SELLER)	Action vendeur

26.2 Verrouillage des prix

Les prix sont **figés** au moment de la commande (snapshot). Cela empêche :

- ▶ La modification des prix par le vendeur après commande
- ▶ Les incohérences entre le montant affiché et le montant payé
- ▶ Les litiges sur les prix

26.3 Vérification du webhook CinetPay

État actuel (v1) : Le webhook valide le format du `cpm_trans_id` et le statut du paiement, mais **ne vérifie pas la signature** de CinetPay.

Planifié : Ajout de la vérification HMAC avec la clé API CinetPay pour garantir l'authenticité des webhooks.

26.4 COD : limite de sécurité

La limite de **75 000 FCFA** pour le COD réduit les risques :

- ▶ Vol lors de la livraison (montant limité)
- ▶ Faux billets (montant vérifiable)
- ▶ Non-paiement par le client (perte limitée)

27. Administration et dashboard

27.1 Statistiques dashboard

```
// GET /api/v1/admin/dashboard
{
  totalUsers: number,
  totalVendors: number,
  totalOrders: number,
  activeOrders: number,      // Non COMPLETED/CANCELLED
  totalDisputes: number,
  openDisputes: number       // status === 'OPEN'
}
```

27.2 Consultation des commandes

```
GET /api/v1/admin/orders?status=PAID&page=1&limit=20
```

L'administrateur peut filtrer les commandes par statut pour identifier :

- ▶ Commandes en attente de confirmation vendeur (PAID)
- ▶ Commandes en livraison (IN_TRANSIT)
- ▶ Litiges ouverts
- ▶ Commandes à problème

27.3 Historique utilisateur

```
GET /api/v1/orders/history?page=1&limit=20
```

Réponse :

```
{
  orders: [...],
  total: 47,
  page: 1,
  totalPages: 3
}
```

28. Tests unitaires

28.1 Tests du service paiement

Fichier : `apps/api/src/modules/payment/payment.service.test.ts`

Test	Description
<code>createEscrow</code>	Crée un escrow avec statut HELD
<code>releaseEscrow</code>	Transition HELD → RELEASED
<code>releaseEscrow (déjà traité)</code>	Erreur si déjà RELEASED/REFUNDED
<code>refundEscrow</code>	Transition HELD → REFUNDED
<code>refundEscrow (introuvable)</code>	Erreur 404 si escrow inexistant

28.2 Tests des routes paiement

Fichier : `apps/api/src/modules/payment/payment.routes.test.ts`

Test	Description
Webhook ACCEPTED	Crée escrow et transition → PAID
Webhook sans trans_id	Erreur 400
GET escrow	Retourne le statut escrow

28.3 Tests du service commande

Fichier : `apps/api/src/modules/order/order.service.test.ts`

Test	Description
<code>selectPaymentMethod (COD)</code>	Transition DRAFT → PAID, paidAt défini
<code>selectPaymentMethod (Mobile)</code>	Transition DRAFT → PENDING_PAYMENT
<code>selectPaymentMethod (COD > 75K)</code>	Erreur ORDER_COD_LIMIT
<code>cancelOrder</code>	Annulation et escrow REFUNDED

28.4 Tests de la machine à états

Fichier : `apps/api/src/modules/order/order.stateMachine.test.ts`

Test	Description
Transitions valides	Chaque transition autorisée passe
Transitions invalides	Chaque transition interdite échoue
États terminaux	COMPLETED et CANCELLED n'ont pas de sortie

29. File d'attente et jobs asynchrones

29.1 Système de queue existant

Le système de file d'attente (`apps/api/src/modules/queue/`) est déjà implémenté pour d'autres jobs :

Job	Description
<code>IMAGE_PROCESS_VARIANTS</code>	Générer les miniatures d'images
<code>CATALOG_AI_IDENTIFY</code>	Identification IA des pièces

29.2 Job planifié pour l'escrow

```
// Nouveau job type : ESCROW_AUTO_RELEASE

// Enqueue lors de la confirmation de livraison
await enqueue('ESCROW_AUTO_RELEASE', {
  orderId: order.id
}, {
  scheduledAt: new Date(Date.now() + 48 * 60 * 60 * 1000), // +48h
  maxAttempts: 3
});
```

29.3 Worker

Le worker poll toutes les **30 secondes** pour les jobs en attente. Il traite les jobs dont `scheduledAt <= now` .

30. État de l'implémentation

30.1 Implémenté (v1)

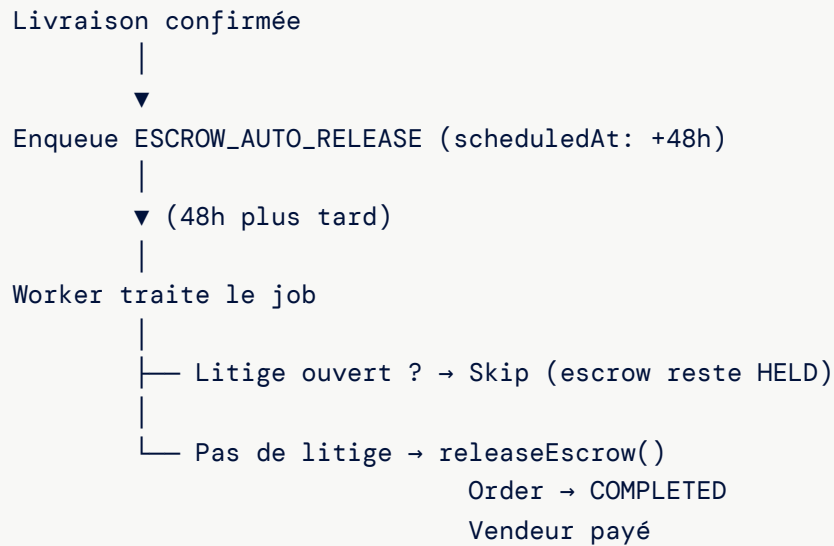
Composant	État	Fichier
Création de commande avec price snapshot	Implémenté	<code>order.service.ts</code>
Machine à états commande	Implémentée	<code>order.stateMachine.ts</code>
Sélection méthode de paiement	Implémentée	<code>order.service.ts</code>
Limite COD 75 000 FCFA	Implémentée	<code>order.service.ts</code>
Intégration CinetPay (initPayment)	Implémentée	<code>lib/cinetpay.ts</code>
Webhook CinetPay	Implémenté	<code>payment.routes.ts</code>
Service escrow (create/release/refund)	Implémenté	<code>payment.service.ts</code>
Consultation escrow	Implémentée	<code>payment.routes.ts</code>
Journal des événements (OrderEvent)	Implémenté	<code>order.service.ts</code>
Lien partagé (shareToken)	Implémenté	<code>order.routes.ts</code>
Validation Zod	Implémentée	<code>validators/order.ts</code>
Tests unitaires (services + routes)	Implémentés	<code>*.test.ts</code>
Mode stub (dev sans CinetPay)	Implémenté	<code>lib/cinetpay.ts</code>

30.2 Non implémenté (planifié)

Composant	Priorité	Description
Auto-libération escrow 48h	Haute	Job schedulé dans la queue
Vérification signature webhook CinetPay	Haute	HMAC avec clé API
Calcul frais de livraison	Moyenne	Basé sur mode et zone
Endpoint de remboursement admin	Moyenne	Admin peut rembourser manuellement
Lien litige → escrow	Moyenne	RESOLVED_BUYER → refundEscrow
Réconciliation vendeur	Basse	Rapports de versement
Notifications paiement WhatsApp	Basse	Câblage avec le module notification

31. Évolutions planifiées

31.1 Phase 2 : Auto-libération escrow



31.2 Phase 2 : Calcul des frais de livraison

Modes de livraison :

- EXPRESS : Livraison < 2h → 5 000 FCFA
- STANDARD : Livraison < 24h → 2 000 FCFA

Supplément zone :

- Zone 1 (centre Abidjan) : +0 FCFA
- Zone 2 (périphérie) : +1 000 FCFA
- Zone 3 (banlieue) : +2 000 FCFA

31.3 Phase 2 : Facturation entreprise

Pour les comptes Enterprise (Phase 2), le paiement pourra être consolidé :

- ▶ Facturation mensuelle au lieu de paiement par commande
- ▶ Délai de paiement 30/60 jours
- ▶ Virement bancaire comme méthode supplémentaire

31.4 Phase 2 : Réconciliation vendeur

Processus mensuel :

1. Lister tous les escrow RELEASED du mois
2. Calculer le montant total par vendeur
3. Déduire la commission Pièces (%)
4. Générer le rapport de versement
5. Effectuer le virement vendeur

32. Référence des fichiers

32.1 Module paiement

Fichier	Chemin	Lignes
Routes	<code>apps/api/src/modules/payment/payment.routes.ts</code>	53
Service	<code>apps/api/src/modules/payment/payment.service.ts</code>	63
Tests routes	<code>apps/api/src/modules/payment/payment.routes.test.ts</code>	123
Tests service	<code>apps/api/src/modules/payment/payment.service.test.ts</code>	77

32.2 Module commande

Fichier	Chemin	Lignes
Routes	<code>apps/api/src/modules/order/order.routes.ts</code>	165
Service	<code>apps/api/src/modules/order/order.service.ts</code>	217
Machine à états	<code>apps/api/src/modules/order/order.stateMachine.ts</code>	23
Tests routes	<code>apps/api/src/modules/order/order.routes.test.ts</code>	216
Tests service	<code>apps/api/src/modules/order/order.service.test.ts</code>	146
Tests state machine	<code>apps/api/src/modules/order/order.stateMachine.test.ts</code>	52

32.3 Intégration CinetPay

Fichier	Chemin	Lignes
Lib CinetPay	<code>apps/api/src/lib/cinetpay.ts</code>	59

32.4 Module livraison

Fichier	Chemin	Lignes
Routes	<code>apps/api/src/modules/delivery/delivery.routes.ts</code>	199
Service	<code>apps/api/src/modules/delivery/delivery.service.ts</code>	137

32.5 Schéma et validateurs

Fichier	Chemin	Contenu
Prisma schema	<code>packages/shared/prisma/schema.prisma</code>	Order, OrderItem, OrderEvent, EscrowTransaction
Validateurs	<code>packages/shared/validators/order.ts</code>	createOrderSchema, confirmOrderSchema, cancelOrderSchema

32.6 Queue

Fichier	Chemin	Contenu
Queue service	<code>apps/api/src/modules/queue/queueService.ts</code>	enqueue, dequeue
Worker	<code>apps/api/src/modules/queue/worker.ts</code>	Poll + handlers

33. FAQ technique

Q : Que se passe-t-il si le webhook CinetPay échoue ?

R : Le serveur retourne toujours 200 OK pour éviter les retries de CinetPay. Si le traitement échoue (escrow non créé), la commande reste en PENDING_PAYMENT. L'administrateur devra vérifier manuellement. Un système de retry est planifié avec la file d'attente pgqueue.

Q : Le vendeur peut-il changer le prix après la commande ?

R : Non. Les prix sont **verrouillés** (snapshot) au moment de la création de la commande. Le champ `priceSnapshot` dans `OrderItem` conserve le prix initial, indépendamment des modifications ultérieures du catalogue.

Q : Que se passe-t-il si le client ne paie pas ?

R : La commande reste en PENDING_PAYMENT. Aucun escrow n'est créé. CinetPay a ses propres délais d'expiration de session. La commande peut être annulée manuellement.

Q : Comment fonctionne le remboursement ?

R : La fonction `refundEscrow()` change le statut de HELD à REFUNDED. En v1, le remboursement effectif (virement retour) nécessite une intervention manuelle via CinetPay. L'automatisation des remboursements est planifiée.

Q : Pourquoi pas d'escrow pour le COD ?

R : Le COD est un paiement direct au livreur. Les fonds ne transitent pas par la plateforme, donc pas besoin de séquestre. La limite de 75 000 FCFA minimise le risque.

Q : Comment vérifier l'état d'un escrow ?

R : Via l'endpoint authentifié :

```
GET /api/v1/orders/{orderId}/escrow
```

Retourne : `{ id, orderId, amount, status, heldAt, releasedAt, refundedAt }`

Q : Peut-on avoir plusieurs escrow pour une même commande ?

R : Non. La relation est 1:1 — `orderId` est unique dans `EscrowTransaction`. Une seule transaction escrow par commande.

Q : Comment sont gérées les commandes multi-vendeurs ?

R : Les `OrderItem` peuvent provenir de différents vendeurs (`vendorId`). Le montant total couvre l'ensemble. En v1, il n'y a qu'un seul escrow pour toute la commande. En Phase 2, le split par vendeur est envisagé pour les versements.

Q : L'auto-libération 48h est-elle active ?

R : Non, pas encore. La machine à états permet la transition DELIVERED → COMPLETED, mais le job schedulé n'est pas implémenté. Actuellement, la libération est manuelle.

Q : Comment tester les paiements en développement ?

R : Sans les variables `CINETPAY_API_KEY` et `CINETPAY_SITE_ID`, la librairie CinetPay passe en **mode stub** et retourne des réponses simulées. Vous pouvez aussi appeler manuellement le webhook avec des payloads de test.

Q : Quel est le montant minimum de commande ?

R : Aucun montant minimum n'est imposé par Pièces. Cependant, CinetPay peut imposer un minimum de transaction (généralement 100 FCFA).